

Guide Setup Local — Projet Final v3

Stack 100% open-source, zéro cloud, zéro carte bancaire

Compagnon technique de `SUJET_Projet_Final_v3_local.md`. Objectif : un correcteur (ou un nouveau membre de groupe) reproduit votre cluster en **< 30 minutes** sur sa machine.

1. Prérequis matériel

Profil	RAM	CPU	Disque libre	Ce qui tourne
 Confortable	≥ 16 Go	6+ cores	30 Go	Stack complète : kind 3 nodes + monitoring + Linkerd + Tempo + Chaos Mesh
 Acceptable	12 Go	4 cores	20 Go	kind 2 nodes, Tempo désactivé , Linkerd OK
 Limite	8 Go	4 cores	15 Go	k3d (plus léger) 1 node, pas de service mesh , monitoring allégé

Vérifier sa RAM disponible :

```
# macOS
sysctl hw.memsize | awk '{print $2/1024/1024/1024 " Go"}'

# Linux
free -h

# Windows (PowerShell)
Get-ComputerInfo | Select-Object TotalPhysicalMemory
```

 Si une seule machine du groupe est confortable (≥ 16 Go), **désignez-la comme machine de démo** et faites pointer ArgoCD / GHCR / Vault dessus.

2. Logiciels à installer

Outil	Version min	Macros d'install
Docker Desktop OU Docker Engine + colima (macOS Apple Silicon)	24+	<code>brew install --cask docker</code> (macOS) / <code>apt install docker.io</code> (Debian)
kubectl	1.30+	<code>brew install kubectl</code>

kind OU k3d	kind $\geq$ 0.23 / k3d $\geq$ 5.6	brew install kind / brew install k3d
helm	3.14+	brew install helm
terraform	1.7+	brew install terraform
argocd CLI	2.10+	brew install argocd
trivy	0.50+	brew install trivy
k9s (<i>optionnel mais très utile</i>)	0.32+	brew install k9s

Vérification rapide :

```
docker version && kubectl version --client && kind version && helm version && \
  terraform version && argocd version --client && trivy version
```

3. Bootstrap du cluster kind (profil confortable)

3.1 — Configuration kind multi-nodes avec port-forward

Créer `kind-config.yaml` :

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: projet-final
networking:
  disableDefaultCNI: false # Calico/Cilium possible en couche 2 si vous voulez
  podSubnet: "10.244.0.0/16"
nodes:
  - role: control-plane
    extraPortMappings:
      - { containerPort: 80, hostPort: 80, protocol: TCP } # Ingress HTTP
      - { containerPort: 443, hostPort: 443, protocol: TCP } # Ingress HTTPS
      - { containerPort: 30080, hostPort: 30080, protocol: TCP } # ArgoCD UI NodePort
      - { containerPort: 30090, hostPort: 30090, protocol: TCP } # Grafana NodePort
  - role: worker
  - role: worker
```

Créer le cluster :

```
kind create cluster --config kind-config.yaml --image kindest/node:v1.30.0
kubectl cluster-info --context kind-projet-final
kubectl get nodes
```

3.2 — Alternative k3d (profil / économe en RAM)

```
k3d cluster create projet-final \
  --servers 1 --agents 2 \
  --port "80:80@loadbalancer" \
  --port "443:443@loadbalancer" \
  --k3s-arg "--disable=traefik@server:0" # on installera notre propre Ingress
```

4. Briques de plateforme à installer (ordre recommandé)

4.1 — Ingress NGINX

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx \
  -n ingress-nginx --create-namespace \
  --set controller.hostPort.enabled=true \
  --set controller.service.type=NodePort
```

4.2 — cert-manager (pour TLS local self-signed)

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  -n cert-manager --create-namespace --set crds.enabled=true
```

4.3 — ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Récupérer le password admin initial
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d ; echo

# Exposer l'UI via NodePort 30080
kubectl -n argocd patch svc argocd-server -p '{"spec":{"type":"NodePort","ports":[{"port":30080}]}'}

# UI ouvrable sur https://localhost:30080
```

4.4 — kube-prometheus-stack (Prometheus + Grafana + Alertmanager)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  --set grafana.service.type=NodePort \
  --set grafana.service.nodePort=30090 \
  --set prometheus.prometheusSpec.retention=2d \
  --set prometheus.prometheusSpec.resources.requests.memory=512Mi \
  --set prometheus.prometheusSpec.resources.limits.memory=1Gi

# Grafana password admin
kubectl -n monitoring get secret kps-grafana -o jsonpath='{.data.admin-password}' | base64
# UI : http://localhost:30090
```

4.5 — Loki + Promtail (logs)

```
helm repo add grafana https://grafana.github.io/helm-charts
helm install loki grafana/loki -n monitoring \
  --set deploymentMode=SingleBinary \
  --set loki.commonConfig.replication_factor=1 \
  --set loki.storage.type=filesystem \
  --set singleBinary.replicas=1

helm install promtail grafana/promtail -n monitoring \
  --set "config.clients[0].url=http://loki:3100/loki/api/v1/push"
```

Ensuite, dans Grafana → Add data source → Loki → URL : `http://loki:3100` .

4.6 — Redpanda (messaging Kafka-compatible)

```
helm repo add redpanda https://charts.redpanda.com
helm install redpanda redpanda/redpanda \
  -n messaging --create-namespace \
  --set statefulset.replicas=1 \
  --set resources.cpu.cores=1 \
  --set resources.memory.container.max=1Gi \
  --set tls.enabled=false \
  --set external.enabled=false
```

Tester :

```
kubectl -n messaging exec -it redpanda-0 -- rpk topic create demo
kubectl -n messaging exec -it redpanda-0 -- rpk topic list
```

4.7 — HashiCorp Vault (dev-mode) + External Secrets Operator

```
# Vault en mode dev (NE PAS UTILISER EN PROD)
helm repo add hashicorp https://helm.releases.hashicorp.com
helm install vault hashicorp/vault \
  -n vault --create-namespace \
  --set "server.dev.enabled=true" \
  --set "server.dev.devRootToken=root"

# External Secrets Operator
helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets external-secrets/external-secrets \
  -n external-secrets --create-namespace --set installCRDs=true
```

Provisionner un secret dans Vault et le synchroniser :

```
# 1. Mettre une valeur dans Vault
kubectl -n vault exec -it vault-0 -- vault kv put secret/api db_password=s3cr3t

# 2. Créer un ClusterSecretStore pointant sur Vault
kubectl apply -f - <<EOF
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
  name: vault-backend
spec:
  provider:
    vault:
      server: "http://vault.vault.svc.cluster.local:8200"
      path: "secret"
      version: "v2"
      auth:
        tokenSecretRef:
          name: vault-token
          namespace: external-secrets
          key: token
EOF

# 3. Provisionner le token dans le namespace external-secrets
kubectl -n external-secrets create secret generic vault-token --from-literal=token=root

# 4. ExternalSecret = bridge Vault → K8s Secret
kubectl apply -f - <<EOF
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: api-secrets
  namespace: app
spec:
  refreshInterval: 30s
  secretStoreRef: { name: vault-backend, kind: ClusterSecretStore }
  target: { name: api-secret }
  data:
    - secretKey: DB_PASSWORD
      remoteRef: { key: api, property: db_password }
EOF
```

4.8 — Linkerd (service mesh léger, mTLS auto)

```
# CLI Linkerd
brew install linkerd

# Pré-check
linkerd check --pre

# Install CRD + control plane
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -

# Vérification
linkerd check

# Injecter le sidecar dans un namespace
kubectl annotate namespace app linkerd.io/inject=enabled
kubectl rollout restart deployment -n app
```

4.9 — Kubecost (FinOps)

```
helm repo add kubecost https://kubecost.github.io/cost-analyzer/
helm install kubecost kubecost/cost-analyzer \
  -n kubecost --create-namespace \
  --set kubecostToken="aGVsbS1jaGFydEBrdWJlY29zdC5jb20=xm343yadf98" \
  --set prometheus.kube-state-metrics.disabled=true \
  --set prometheus.nodeExporter.enabled=false

# Accéder à l'UI
kubectl -n kubecost port-forward svc/kubecost-cost-analyzer 9090:9090
# Ouvrir http://localhost:9090
```

4.10 — MailHog (récepteur SMTP pour les alertes Alertmanager)

```
kubectl create namespace mail
kubectl -n mail apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata: { name: mailhog }
spec:
  replicas: 1
  selector: { matchLabels: { app: mailhog } }
  template:
    metadata: { labels: { app: mailhog } }
    spec:
      containers:
        - name: mailhog
          image: mailhog/mailhog:latest
          ports:
            - { containerPort: 1025 }    # SMTP
            - { containerPort: 8025 }    # UI
---
apiVersion: v1
kind: Service
metadata: { name: mailhog, namespace: mail }
spec:
  selector: { app: mailhog }
  ports:
    - { name: smtp, port: 1025, targetPort: 1025 }
    - { name: http, port: 8025, targetPort: 8025 }
EOF
```

Configurer Alertmanager pour utiliser MailHog :

```
receivers:
- name: mailhog
  email_configs:
  - to: 'team@projet-final.local'
    from: 'alerts@projet-final.local'
    smarthost: 'mailhog.mail.svc.cluster.local:1025'
    require_tls: false
    send_resolved: true
```

4.11 — Bonus : Tempo (traces) + Chaos Mesh + Knative

```
# Tempo (lourd, profil 🟢 uniquement)
helm install tempo grafana/tempo -n monitoring \
  --set tempo.storage.trace.backend=local

# Chaos Mesh
helm repo add chaos-mesh https://charts.chaos-mesh.org
helm install chaos-mesh chaos-mesh/chaos-mesh \
  -n chaos-mesh --create-namespace --version 2.6.3

# Knative (Serving + Eventing)
kubectl apply -f https://github.com/knative/serving/releases/download/knative-v1.14.0/serving.yaml
kubectl apply -f https://github.com/knative/serving/releases/download/knative-v1.14.0/serving.yaml
kubectl apply -f https://github.com/knative/net-kourier/releases/download/knative-v1.14.0/net-kourier.yaml
```

5. CI/CD : GitHub Actions → GHCR → repo gitops → ArgoCD

5.1 — Authentification GHCR

Dans **chaque repo GitHub public** du groupe :

1. Settings → Actions → General → **Workflow permissions** = Read and write .
2. Le token `${{ secrets.GITHUB_TOKEN }}` permet de pusher sur `ghcr.io/<owner>/<image>` .

5.2 — Workflow type (`.github/workflows/ci.yaml`)

```

name: CI
on:
  push: { branches: [main] }

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }/api

jobs:
  build-test-scan:
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write, security-events: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

      - name: Lint + tests
        run: |
          cd services/api
          npm ci && npm run lint && npm test

      - name: Build & push image
        uses: docker/build-push-action@v5
        with:
          context: services/api
          push: true
          tags: |
            ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }
            ${ env.REGISTRY }/${ env.IMAGE_NAME }:latest

      - name: Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }
          severity: HIGH,CRITICAL
          exit-code: '1' # ⚠️ obligatoire pour bloquer la pipeline

      - name: Bump tag in gitops repo
        run: |
          git clone https://x-access-token:${ secrets.GITOPS_PAT }@github.com:${ github
          cd projet-final-gitops
          sed -i "s|image: .*api:.*|image: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ g
          git -c user.email=ci@bot -c user.name=CI commit -am "chore: bump api ${ github.
          git push

```

GITOPS_PAT = un *fine-grained personal access token* GitHub avec droit `contents:write` sur le repo gitops uniquement. C'est l'équivalent de Workload Identity Federation pour notre stack locale.

5.3 — ArgoCD App of Apps

```
# bootstrap/root-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: root, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/<org>/projet-final-gitops
    targetRevision: main
    path: apps
  destination: { server: https://kubernetes.default.svc, namespace: argocd }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

6. Démo Self-Heal en 30 secondes

```
# 1. Dans un terminal : suivre le Deployment
kubectl get deploy/api -n app -w

# 2. Dans un second : casser intentionnellement
kubectl scale deploy/api -n app --replicas=0

# 3. Attendre ~90s – ArgoCD repasse à replicas=1 automatiquement
# (visible aussi dans l'UI ArgoCD : OutOfSync → Syncing → Synced)
```

7. Démo Canary avec Argo Rollouts

```
# Installer Argo Rollouts
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/latest

# Plugin kubectl
brew install argoproj/tap/kubectl-argo-rollouts
```

Manifeste Rollout exemple :

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: api, namespace: app }
spec:
  replicas: 4
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: { duration: 2m }
        - setWeight: 50
        - pause: { duration: 2m }
        - setWeight: 100
      analysis:
        templates: [{ templateName: success-rate }]
  selector: { matchLabels: { app: api } }
  template: # ... cf. doc Argo Rollouts

```

Démo live :

```
kubectl argo rollouts get rollout api -n app --watch
```

8. Troubleshooting fréquent

Symptôme	Cause probable	Fix
kind: failed to create cluster: ERROR: failed to ensure docker network	Conflit Docker network	<code>docker network prune</code> puis <code>kind delete cluster</code> + recréer
Pods Pending indéfiniment	RAM insuffisante (kind node NotReady)	Réduire requests/limits, désactiver Tempo, passer en k3d
kubectl: connection refused après reboot	kind ne redémarre pas tout seul	<code>docker start projet-final-control-plane projet-final-worker projet-final-worker2</code>
Image GHCR denied: permission_denied	Repo privé OU image privée par défaut	GitHub → packages → image → Visibility = public
ArgoCD OutOfSync permanent	sync waves mal ordonnés	Annoter : <code>argocd.argoproj.io/sync-wave: "0"</code> infra, "1" monitoring, "2" apps

Linkerd <code>linkerd-proxy: 502</code>	sidecar pas injecté	<code>kubectl annotate ns app linkerd.io/inject=enabled</code> puis <code>kubectl rollout restart deploy -n app</code>
Trivy passe quand même avec CRITICAL	<code>continue-on-error: true</code> activé	Le retirer; mettre <code>exit-code: '1'</code> dans <code>trivy-action</code>
Grafana ne voit pas Loki	Datasource URL fausse	URL Loki interne : <code>http://loki.monitoring.svc.cluster.local:3100</code>
<code>kubectl port-forward</code> se coupe	Time out réseau Wi-Fi	Encapsuler dans une boucle : <code>while true; do kubectl port-forward ...; done</code>

9. Nettoyage en fin de soutenance

```
# Détruire le cluster (libère ~9 Go RAM immédiatement)
kind delete cluster --name projet-final

# OU si k3d
k3d cluster delete projet-final

# Optionnel : purger les images Docker qui ne servent plus
docker system prune -a --volumes
```

💡 **Geste valorisé** (+1 pt) : faire le `kind delete cluster` devant le jury à la fin de la démo.

10. Commandes-clés à mémoriser

```
# État global
kubectl get pods -A | grep -v -E "Running|Completed" # doit être vide
kubectl top nodes && kubectl top pods -A | head -20 # consommation
kubectl get netpol -A # network policies

# ArgoCD
argocd app list
argocd app sync <app>
argocd app get <app> --hard-refresh

# Messaging (Redpanda)
kubectl -n messaging exec -it redpanda-0 -- rpk topic list
kubectl -n messaging exec -it redpanda-0 -- rpk topic consume <topic> --num 5

# Vault
kubectl -n vault exec -it vault-0 -- vault kv list secret/
kubectl -n vault exec -it vault-0 -- vault kv get secret/<key>

# Argo Rollouts
kubectl argo rollouts list rollout -A
kubectl argo rollouts promote <name> -n <ns>

# Linkerd
linkerd viz dashboard &
linkerd viz top -n app

# Chaos Mesh (bonus)
kubectl apply -f chaos/podchaos-kill-api.yaml
kubectl -n app describe podchaos kill-api
```

11. Estimation de consommation (profil confortable)

Brique	RAM idle	RAM en charge
kind 3 nodes	1.2 Go	1.5 Go
3 microservices x 2 replicas	0.6 Go	1.2 Go
Redpanda 1 broker	0.4 Go	0.7 Go
PostgreSQL	0.3 Go	0.5 Go
ArgoCD	0.8 Go	1.0 Go
kube-prometheus-stack	1.5 Go	2.0 Go
Loki + Promtail	0.3 Go	0.5 Go
Linkerd	0.4 Go	0.6 Go

Vault + External Secrets	0.2 Go	0.3 Go
Kubecost	0.4 Go	0.6 Go
Total	6.1 Go	8.9 Go
+ Tempo (bonus)	+0.8 Go	+1.0 Go
+ Chaos Mesh (bonus)	+0.3 Go	+0.4 Go

→ Sur une machine 16 Go RAM : marge confortable. → Sur une machine 12 Go RAM : virer Tempo + Chaos Mesh + Knative. → Sur une machine 8 Go RAM : k3d 1 node + pas de service mesh + Loki en mode minimal.

— A. Benhamdi, juin 2026